11 Objektově orientované programování v PHP

V této kapitole se dozvíte:

- Jak vytvářet aplikace v PHP pomocí objektového přístupu.
- Jak vytvořit odvozenou třídu.
- Jak přetěžovat proměnné a metody.

Po jejím prostudování byste měli být schopni:

- Umět vytvořit třídu v PHP.
- Umět použít dědičnost při tvorbě tříd.
- Umět přetěžovat proměnné a metody.

Klíčová slova této kapitoly:

class, konstruktor, destruktor, dědičnost, public, protected, private.

Doba potřebná ke studiu: 2 hodiny

Průvodce studiem



Studium této kapitoly předpokládá alespoň základní znalosti objektově orientovaného programování. Kapitola se nezabývá vysvětlováním principů objektově orientovaného programování, ale vysvětluje tvorbu tříd v jazyce PHP.

Vývoj programů byl v minulosti realizován pomocí strukturovaného programování, které se vyznačuje tím, že striktně odděluje data a funkce. Během 90. let 20. století se začalo rozšiřovat objektově orientované programování (OOP). V tomto přístupu jsou základním stavebním kamenem všech programů objekty, které obsahují jak data, tak funkce. Definici typu objektu označujeme jako třídu, objekty jsou pak jednotlivé instance této třídy. Jako jednoduchý příklad uveďme situaci, kdy bychom pracovali s geometrickými tvary. Jedním ze základních geometrických tvarů je obdélník. K reprezentaci obdélníku vytvoříme třídu *Obdélník*. Data v této třídě budou tvořit rozměry obdélníku, tj. jeho *šířka* a *výška*. Kromě toho můžeme ve třídě definovat funkce (metody), které vypočítají *obvod* a *obsah* obdélníku. Vytvořením instance třídy vytvoříme jeden konkrétní obdélník o zadaných rozměrech.

V jazyce PHP verze 5 je již k dispozici kompletní podpora objektově orientovaného programování. Používání objektového přístupu ale není v PHP nutností, jako je tomu například v jazyce Java. Pokud nechceme, můžeme používat klasický strukturovaný přístup. Volba přístupu při tvorbě aplikace závisí na tom, jak složitá je naše aplikace. Pokud vytváříme pouze jednoduchou webovou aplikaci skládající se z několika PHP skriptů, je použití

objektového přístupu zbytečné. V případě rozsáhlých aplikací, které se stále rozšiřují je již použití objektů téměř nezbytné.

Tato kapitola se nebude zabývat principy OOP, ale bude se zabývat syntaxí v PHP, která souvisí s tvorbou tříd, objektů apod.

11.1 Vytvoření třídy a její instance



Pokud používáme při tvorbě webové aplikace v PHP objektový přístup, bude naše aplikace obsahovat definici tříd, vytvoření instancí tříd a volání jejich metod. Aby byly zdrojové kódy aplikace přehledné, je vhodné vytvořit pro každou třídu samostatný soubor. Pokud potřebujeme v nějaké skriptu použít určitou třídu, stačí vložit do tohoto skriptu soubor obsahující definici třídy. Definice třídy začíná klíčovým slovem `class`, za kterým následuje její název. Za názvem může následovat definice rodičovských objektů. Ta se zde ale vyskytuje pouze v případě, že chceme, aby objekt zdědil vlastnosti nadřazené třídy. Dědičnost (inheritance) je jedním ze základních pilířů OOP – nové třídy můžeme vytvářet rozšiřováním existujících tříd. Pokud chceme při definici třídy použít nadřazenou třídu, použijeme k tomu klíčové slovo `extends`. Dále již následují složené závorky, ve kterých je vlastní definice třídy.

Definice třídy se skládá ze dvou částí. První částí je definování členských proměnných třídy, druhou částí je definování jejích metod. Definice metody začíná klíčovým slovem `function`, stejně jako definice funkcí. V části obsahující metody mohou být umístěny dvě speciální metody – konstruktor a destruktor.

Konstruktor je metoda, která se automaticky volá při vytváření instance třídy. V konstruktoru je možné použít parametry (stejně jako v jakékoliv jiné metodě), které se předávají vytvořenému objektu. Od PHP verze 5 má konstruktor jednotný název `__construct()` – před slovem construct je uveden 2x za sebou znak _ (podržítko). V nižších verzích PHP se při definici konstruktoru používal stejný název jako je název třídy. Pokud vytváříme třídu, která dědí z jiné třídy, konstruktor nadřazené třídy se nevolá automaticky. Je nutné jej v případě potřeby zavolat, k čemuž slouží příkaz `parent::__construct()`.

Destruktor je metoda, která se volá při odstraňování objektu. Definice destruktoru není nutná, protože PHP automaticky provádí správu paměti. Doba běhu skriptů je většinou krátká, takže není potřeba odstraňovat objekty ručně.

Pokud ale i přesto potřebujeme destruktor vytvořit, použijeme metodu `__destruct()` – před slovem destruct je opět 2x znak _ (podtržítko).

Instanci třídy vytvoříme pomocí klíčového slova `new`, za kterým uvedeme název vytvářené třídy. Pokud konstruktor obsahuje parametry, je potřeba je uvést při vytváření instance třídy. Pokud je konstruktor bez parametrů, není nutné uvádět za názvem třídy prázdné kulaté závorky.

Následující výpis obsahuje ukázku vytvoření třídy a jejích potomků. Třída `OdvozenaTrida1` neobsahuje v konstruktoru volání nadřazené třídy, takže text „`Základní třída`“ se nevypíše. V konstruktoru třídy `OdvozenaTrida2` je již volání konstruktoru nadřazené třídy uvedeno, takže se vypíše i text „`Základní třída`“. Obrázek pod výpis pak zobrazuje výsledek zobrazený v prohlížeči.

Výpis 11.1 Vytvoření třídy.

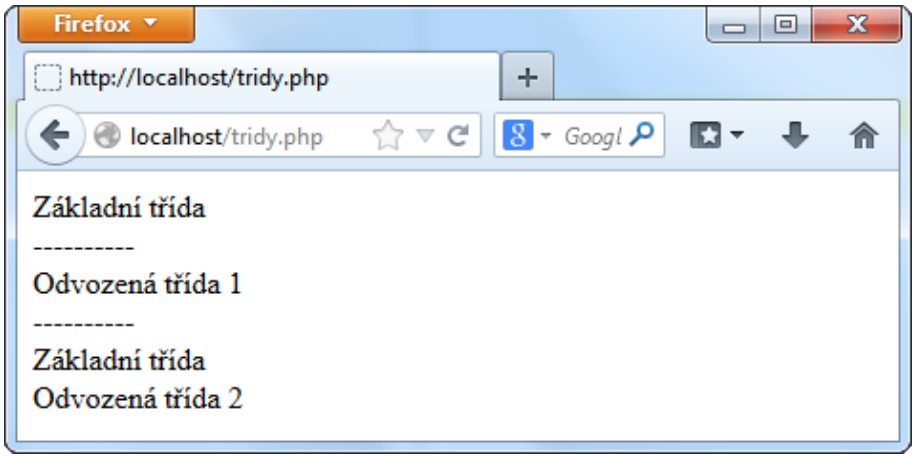
```
<?php
  class ZakladniTrida {
    function __construct() {
      echo "Základní třída<br>";
    }
  }

  class OdvozenaTrida1 extends ZakladniTrida {
    function __construct() {
      echo "Odvozená třída 1<br>";
    }
  }

  class OdvozenaTrida2 extends ZakladniTrida {
    function __construct() {
      parent::__construct();
      echo "Odvozená třída 2<br>";
    }
  }

  $objekt = new ZakladniTrida();
  echo "----------<br>";
  $objekt = new OdvozenaTrida1();
  echo "----------<br>";
  $objekt = new OdvozenaTrida2();
?>
```





Obrázek 11.1 Vytvoření třídy.

11.2 Členské proměnné a metody, zapouzdření tříd

Definice každé třídy může obsahovat proměnné nebo metody (funkce), které bývají nazývány také jako členské proměnné a členské metody. Po vytvoření instance objektu můžeme přistupovat k členským proměnným nebo metodám pomocí operátoru `->`. Při přístupu k členské proměnné se za operátorem `->` již neuvádí znak `$` (dolar). Pokud potřebujeme v rámci některé členské metody pracovat s aktuálním objektem, použijeme k tomu proměnnou `$this`, která je k dispozici v každé třídě a představuje odkaz na aktuální objekt.



Dalším pilířem objektově orientovaného programování je zapouzdření (encapsulation). Pomocí zapouzdření můžeme při definici třídy určit, komu jsou přístupné její členské proměnné a metody. V jazyce PHP existují tři možnosti definice přístupu, které jsou stejné jako ve většině jiných programovacích jazyků. Jedná se o přístup `private`, `protected` a `public`. Přístup `private` znamená, že k proměnné nebo k metodě smí přistupovat pouze třída, ve které jsou definovány. Přístup `protected` znamená, že k proměnné nebo metodě smí přistupovat třída, ve které jsou definovány, a nebo potomci třídy (odvozené třídy). Přístup `public` znamená, že k proměnné nebo metodě smí přistupovat kdokoliv. Následující výpis zobrazuje ukázku použití různých přístupových práv k proměnným.

Výpis 11.2 Ukázka použití private, protected a public.



```
<?php
  class Trida {
    private $priv = 1;
    protected $prot = 2;
    public $publ = 3;
```

```
  }

  $o = new Trida();
  echo $o->publ; // v poradku
  echo $o->prot; // chyba
  echo $o->priv; // chyba
?>
```

S přístupem k členským proměnným a metodám souvisí statické proměnné a metody. Statické proměnné nebo metody můžeme používat aniž bychom vytvořili instanci třídy. Pokud chceme označit některou z členských proměnných nebo metod jako statickou, uvedeme před název proměnné nebo metody klíčové slovo `static`. Přístup ke statickým proměnným nebo metodám provádíme pomocí operátoru `::` (znak dvojtečky 2x za sebou). Ve statických metodách nelze použít proměnnou `$this`. Pokud potřebujeme ve statické metodě pracovat s aktuálním objektem, je potřeba použít popisovat `self::`.



V některých případech je žádoucí, aby potomci třídy nepřepisovali některé z metod. Takové metody označíme jako finální, a to pomocí klíčového slova `final`. Pokud označíme jako finální celou třídu, nelze vytvořit jejího potomka. Přepsání konečné metody nebo vytvoření potomka konečné třídy vede ke kritické chybě. Následující výpis ukazuje použití statických proměnných nebo metod a konečných metod nebo tříd.

Výpis 11.3 Statické proměnné a metody, konečné metody.

```
<?php
  class Trida {
    public static $cislo = 10;
    public static function tiskCisla() {
      echo "Číslo: " . self::$cislo;
    }
  }

  echo Trida::$cislo . "<br>";
  Trida::tiskCisla();

  class TridaFinal {
    private $cislo = 20;
    public final function tiskCisla() {
      echo "Číslo: " . $this->cislo;
    }
  }
```



```
  class TridaFinal2 extends TridaFinal {
    public function tiskCisla2() {  // zpusobi chybu
      echo "Číslo: " . $this->cislo;
    }
  }
?>
```

11.3 Přetěžování proměnných a metod

Pojem přetěžování má v PHP jiný význam než je obvyklé. Jako přetěžování bývá v programovacích jazycích označován postup, kdy jedna metoda se stejným názvem existuje v různých variantách, které se liší svými parametry (jejich počtem nebo datovým typem) nebo svou návratovou hodnotou.

Přetěžování v PHP poskytuje způsob jak dynamicky vytvářet členské proměnné a metody. K tomuto účelu jsou určeny speciální funkce. Přetěžující metody se použití v případě, že pracujeme s proměnnými nebo metodami, které zatím nebyly vytvořeny nebo nejsou v aktuálním místě přístupné (např. proto, že jsou definovány jako `private`). Všechny přetěžující metody musí být definovány jako public.



Pro přetěžování proměnných jsou důležité dvě funkce, a to `__set()` a `__get()`. Stejně jako v případě metod konstruktoru a destruktoru začíná název metody dvěma znaky _ (podtržítka) za sebou. Funkce `__set()` slouží k nastavení hodnoty a má dva parametry. Prvním je název proměnné, druhým hodnota, kterou do ní chceme uložit. Funkce nic nevrací, resp. vrací typ `void`. Funkce `__get()` má pouze jeden parametr a tím je název proměnné, jejíž hodnotu chceme získat. Metoda vrací obsah proměnné, která může být různého typu, takže ani návratový typ není pěvně dán.

Jestliže jako první parametr metody `__set()` použijeme název proměnné, která neexistuje, je tato proměnná vytvořena. Metody `__get()` a `__set()` je možné použít pouze při práci s objektem. Nelze je použít na statické proměnné a nemohou být deklarovány jako static.

Výpis 11.4 Použití metod __get() a __set().



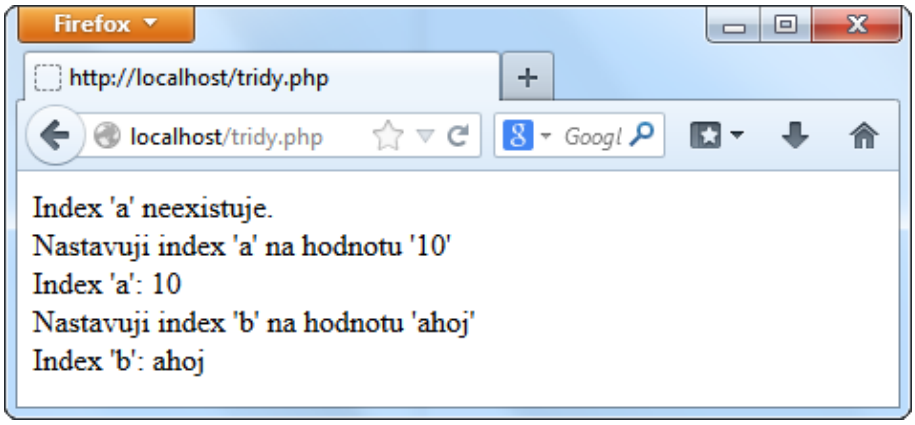
```
<?php
  class Trida {
    private $data = array();

    public function __set($name, $value) {
```

```
      echo "Nastavuji index '$name' na hodnotu
       '$value'<br>";
      $this->data[$name] = $value;
    }

    public function __get($name) {
      if (array_key_exists($name, $this->data)) {
        echo "Index '$name': ";
        return $this->data[$name];
      }
      else {
        echo "Index '$name' neexistuje.";
      }
    }
  }

  // pri vytvareni objektu nemusi byt uvedeny kulate
   zavorky
  $test = new Trida;
  echo $test->a . "<br>";
  $test->a = 10;
  echo $test->a . "<br>";
  $test->b = "ahoj";
  echo $test->b . "<br>";
?>
```



Obrázek 11.2 Použití metod __get() a __set().

Jazyk PHP umožňuje přetěžovat nejen proměnné, ale také metody. Pro přetěžování metod jsou v PHP určeny dvě metody - `__call()` a `__callStatic()`. Metoda `__call()` slouží pro přetěžování nestatických metod, metoda `__callStatic()` pro statické metody. Obě metody mají dva parametry. Prvním je název metody a druhým je pole, ve kterém jsou uloženy parametry metody.



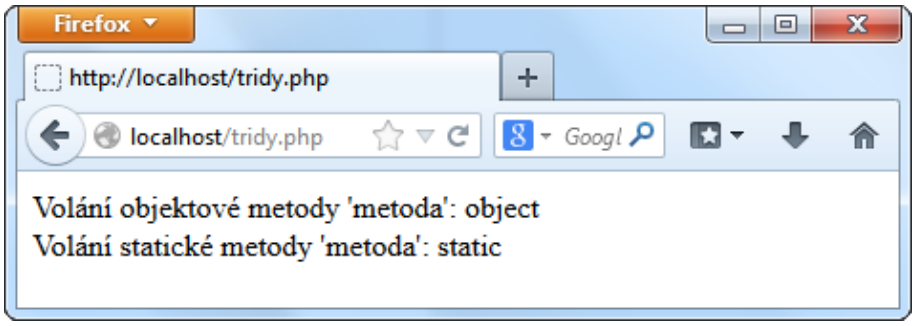
Výpis 11.5 Použití metod __call() a __callStatic().



```
<?php
  class Trida {
    public function __call($name, $arguments) {
      echo "Volání objektové metody '$name': " .
       implode(", ", $arguments) . "<br>";
    }

    public static function __callStatic($name,
     $arguments) {
      echo "Volání statické metody '$name': " .
       implode(', ', $arguments). "<br>";
    }
  }

  $arg = array(100, 200, 300, 400);
  $test = new Trida;
  $test->metoda('object');
  Trida::metoda('static');
?>
```



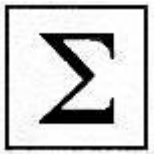
Obrázek 11.3 Použití metod __call() a __callStatic().



Kontrolní otázky:

1. Jakým vytvoříme třídu v PHP?
2. Jaký je rozdíl mezi `public`, `protected` a `private`?
3. Jak je možné v PHP vytvořit konstruktor a jak destruktor?
4. Co je to přetěžování proměnných?
5. Co je to přetěžování metod?

Shrnutí obsahu kapitoly



V této kapitole jste se seznámili s tvorbou tříd v jazyce PHP. Třídy mohou obsahovat členské proměnné a členské metody. Speciálními metodami jsou konstruktor a destruktor, které slouží k vytvoření nebo zrušení instance třídy. Přístup ke členským proměnným nebo metodám může být nastaven jako `public`, `protected` nebo `private`. Jazyk PHP umožňuje dynamicky vytvářet členské proměnné nebo metody. Tato činnost bývá označována jako přetěžování.